



Université Côte d'Azur

EUR Sciences du Vivant et de la Santé

Master's degree in Bioinformatics and Computational Biology

Second Year (M2) - Academic Year: 2025–2026

**Compound Discoverer Refinement and
Metabolomics Pipeline:
Step-by-Step Tutorial**

Ianca Karine PRUDENCIO DE ALBUQUERQUE

Supervisors: Hélène MARIE, PhD

Océane PIERROT, MSc



Team: Pathophysiology of Neural Circuits and Behavior

Institut de Pharmacologie Moléculaire et Cellulaire (IPMC)

660 route des Lucioles Sophia Antipolis

06560, Valbonne

LIST OF TABLES

Table I. Main project folders and their purpose.	22
Table II. Main Shiny application files.	23
Table III. Helper scripts used by the CD-MetaboRefine Shiny application.	23
Table IV. Modular organization of the CD-MetaboRefine pipeline.	26
Table V. Auxiliary scripts included in the project	27
Table VI. Input files used by CD-MetaboRefine.	28
Table VII. Recommended metadata columns.	29
Table VIII. Example metadata table.	29
Table IX. Example injection order file.	31
Table X. Main output files generated by CD-MetaboRefine.	47
Table XI. Sample name matching examples.	56

LIST OF FIGURES

Figure 1. CRAN download page showing the Windows installer for R.	13
Figure 2. Posit download page used to download RStudio Desktop.	14
Figure 3. Downloading CD-MetaboRefine from GitHub using Code → Download ZIP.	15
Figure 4. Repository structure of the metabolomics pipeline.	18
Figure 5. Compound Discoverer Input Files tab showing raw files, sample type, batch information, and experimental metadata columns that can be exported to Excel and used to prepare the injection order file.	32
Figure 6. CD-MetaboRefine main interface after launching the Shiny application.	36
Figure 7. Input file selection and output directory configuration in CD-MetaboRefine.	37
Figure 8. Settings Builder run setup options, including statistical analysis, normalization, filtering, and duplicate handling.	39
Figure 9. Visual output settings for heatmaps, volcano plots, and PCA plots.	40
Figure 10. Export level configuration in the Settings Builder.	41
Figure 11. Pipeline Log tab before pipeline execution.	41
Figure 12. Results Gallery tab displaying generated summaries and visual outputs after pipeline execution.	42
Figure 13. Example visual outputs generated by CD-MetaboRefine. (A) PCA plot used to inspect sample distribution; (B) volcano plot used to visualize statistical significance and fold change; (C) heatmap used to compare feature abundance patterns across samples; and (D) normalization comparison plot used to inspect preprocessing and QC-based correction methods.	52

TABLE OF CONTENTS

1. INTRODUCTION.....	7
2. REQUIREMENTS.....	9
2.1 SOFTWARE REQUIREMENTS.....	9
2.2 PROJECT REQUIREMENTS.....	9
2.3 R PACKAGE REQUIREMENTS.....	10
2.4 INPUT FILE REQUIREMENTS.....	10
2.5 SUPPORTED INPUT FORMATS.....	10
2.6 RECOMMENDED OPERATING SYSTEMS.....	11
3. INSTALL R AND Rstudio.....	12
3.1 HOW TO INSTALL R.....	12
3.2 HOW TO INSTALL RSTUDIO DESKTOP.....	13
3.3 RECOMMENDED CHECK AFTER INSTALLATION.....	14
4. DOWNLOAD THE PROJECT.....	15
4.1 OPTION A — DOWNLOAD AS ZIP.....	15
4.2 OPTION B — CLONE WITH GIT.....	16
4.3 AFTER DOWNLOADING THE PROJECT.....	17
5. OPEN THE PROJECT IN RStudio.....	19
5.1 OPEN THE PROJECT USING THE .RPROJ FILE.....	19
5.2 WHY THIS STEP IS IMPORTANT.....	19
5.3 IF NO .Rproj FILE IS AVAILABLE.....	19
6. INSTALL REQUIRED PACKAGES.....	21
6.1 INSTALL THE STARTUP PACKAGES.....	21
6.2 AUTOMATIC INSTALLATION OF PIPELINE DEPENDENCIES.....	21
6.3 OPTIONAL FULL MANUAL INSTALLATION.....	22
6.4 WHY PACKAGE INSTALLATION MAY TAKE TIME.....	22
7. APPLICATION AND PIPELINE STRUCTURE.....	23
7.1 MAIN PROJECT FOLDERS.....	23
7.2 SHINY APPLICATION FILES.....	24
7.3 APPLICATION HELPER SCRIPTS.....	24
7.4 MAIN APPLICATION TABS.....	25
7.4.1 Data Overview.....	25
7.4.2 Settings Builder.....	25
7.4.3 Pipeline Log.....	25
7.4.4 Results Gallery.....	26
7.5 PIPELINE CONFIGURATION FILES.....	26
7.6 PIPELINE ANALYSIS MODULES.....	27

7.7 MAIN EXECUTION SCRIPT.....	27
7.8 AUXILIARY SCRIPTS.....	28
8. PREPARE THE INPUT FILES.....	29
8.1 COMPOUND DISCOVERER FEATURE TABLE.....	29
8.2 METADATA FILE.....	30
8.3 INJECTION ORDER FILE.....	31
8.4 REFERENCE OR COMPARISON FILE.....	33
8.5 GOOD PRACTICES BEFORE RUNNING THE PIPELINE.....	33
9. LAUNCH THE APPLICATION AND RUN THE PIPELINE.....	35
9.1 STEP 1 — LAUNCH CD-METABOREFINE.....	35
9.2 STEP 2 — CHECK PACKAGE AVAILABILITY.....	36
9.3 STEP 3 — UPLOAD THE INPUT FILES.....	36
9.4 STEP 4 — SELECT METADATA GROUPS.....	37
9.5 STEP 5 — CONFIGURE THE OUTPUT DIRECTORY.....	38
9.6 STEP 6 — REVIEW THE SETTINGS BUILDER.....	38
9.7 STEP 7 — RUN THE PIPELINE.....	41
9.8 STEP 8 — INSPECT THE PIPELINE LOG.....	41
9.9 STEP 9 — INSPECT THE RESULTS GALLERY.....	42
9.10 STEP 10 — OPEN THE OUTPUT FOLDER.....	43
10. RUN THE PIPELINE FROM CODE.....	44
10.1 STEP 1 — CREATE THE LOCAL SETTINGS FILE.....	44
10.2 STEP 2 — CONFIGURE THE MAIN OPTIONS.....	45
10.3 STEP 3 — EXECUTE THE PIPELINE FROM RSTUDIO.....	45
10.4 STEP 4 — EXECUTE THE PIPELINE FROM A TERMINAL.....	45
10.5 STEP 5 — READ THE PIPELINE LOG.....	46
10.6 STEP 6 — INSPECT THE GENERATED OUTPUTS.....	46
10.7 WHEN TO USE THE SCRIPT-BASED MODE INSTEAD OF THE SHINY APP.....	46
11. UNDERSTAND THE OUTPUT STRUCTURE.....	48
11.1 MAIN OUTPUT FILES.....	48
11.2 TYPICAL OUTPUT ORGANIZATION.....	49
11.3 RECOMMENDED FILES TO REVIEW FIRST.....	49
11.4 AUDIT FILES.....	50
11.5 STATISTICAL RESULT TABLES.....	50
11.6 PCA PLOTS.....	51
11.7 VOLCANO PLOTS.....	51
11.8 HEATMAPS.....	51
11.9 NORMALIZATION AND QC DIAGNOSTIC PLOTS.....	52
11.10 EXAMPLE VISUAL OUTPUTS.....	52

11.11 IMPORTANT INTERPRETATION NOTE.....	53
12. TROUBLESHOOTING.....	54
12.1 THE APPLICATION DOES NOT OPEN.....	54
12.2 MISSING PACKAGES.....	54
12.3 settings.R NOT FOUND.....	55
12.4 FILE READ ERROR.....	56
12.5 SAMPLE NAMES DO NOT MATCH.....	56
12.6 MISSING METADATA COLUMNS.....	57
12.7 WEIGHT NORMALIZATION ERROR.....	58
12.8 QC-BASED NORMALIZATION DOES NOT WORK.....	58
12.9 NO OUTPUT FILES WERE GENERATED.....	59
12.10 RESULTS GALLERY IS EMPTY.....	59
12.11 RScript NOT RECOGNIZED.....	60
12.12 GENERAL DEBUGGING CHECKLIST.....	60

1. INTRODUCTION

This tutorial provides a step-by-step guide for using **CD-MetaboRefine**, the Shiny interface of the **Compound Discoverer Refinement and Metabolomics Pipeline**.

CD-MetaboRefine is an R/Shiny application developed to support the processing, refinement, normalization, statistical analysis, visualization, and export of untargeted metabolomics data generated from **Compound Discoverer**.

The workflow was designed for analyses in which Compound Discoverer feature tables need to be combined with experimental metadata, quality-control information, normalization procedures, filtering criteria, duplicate handling, statistical testing, and graphical outputs.

The project can be used through two complementary execution modes:

1. CD-MetaboRefine Shiny application

A graphical interface designed for interactive use. This mode is recommended for users who prefer to upload files, review settings, run the pipeline, inspect logs, and explore results through a visual interface.

2. Script-based pipeline

A reproducible execution mode based on R scripts. This mode is recommended for advanced users, debugging, batch processing, and analyses that require full control of the configuration files.

By following this tutorial, users will learn how to:

1. install the required software and R packages;
2. download and open the project correctly;
3. prepare Compound Discoverer feature tables and metadata files;
4. upload input files using CD-MetaboRefine;
5. configure normalization, filtering, duplicate handling, statistical analysis, PCA, volcano plots, heatmaps, and export options;
6. run the workflow using either the Shiny application or the script-based pipeline;
7. inspect quality-control summaries, normalization diagnostics, PCA plots, volcano plots, heatmaps, statistical result tables, and MetaboAnalyst-compatible export files.

The pipeline integrates metabolomics feature tables with experimental metadata to generate structured outputs, including:

- cleaned and normalized assay matrices;
- metadata alignment reports;
- filtering and quality-control audit files;
- duplicate handling reports;
- normalization diagnostic plots;
- PCA visualizations;
- statistical analysis tables;

- volcano plots;
- heatmaps;
- MetaboAnalyst-ready export files;
- execution logs and output summaries.

The main goal of CD-MetaboRefine is to make untargeted metabolomics data processing more reproducible, transparent, and accessible, while still allowing advanced users to adjust the analysis parameters according to the experimental design.

2. REQUIREMENTS

Before using **CD-MetaboRefine**, make sure that the required software, project files, and input data files are available on your computer.

2.1 SOFTWARE REQUIREMENTS

The following software is required:

1. **R**

R is the programming language used by the pipeline. This tutorial was prepared and tested using R version 4.5.3. Users are encouraged to install a recent stable version of R from CRAN.

2. **RStudio Desktop**

RStudio is recommended for opening the project, launching the Shiny application, editing configuration files, and running the script-based pipeline.

3. **Internet connection**

An internet connection is recommended during the first setup because some R packages may need to be downloaded and installed.

4. **Git**

Git is optional. It is only required if the user wants to clone the repository directly from GitHub instead of downloading the project as a ZIP file.

2.2 PROJECT REQUIREMENTS

The project folder must be downloaded and extracted before use.

Do not run the project directly from a compressed ZIP archive. The complete folder structure must be preserved because the application and the pipeline use relative paths to locate scripts, configuration files, images, and output folders.

The main project components are:

- *app.R*, used to launch the CD-MetaboRefine Shiny application;
- *app/*, containing the Shiny interface files;
- *pipeline/*, containing the modular R scripts used for data processing and analysis;
- *pipeline/config/*, containing *settings.example.R*, the configuration template used to create local analysis settings;
- *images/*, containing example images used by the documentation and interface;
- *data/*, which may contain example or local input files;
- *output/*, generated automatically after running the workflow.

A local *settings.R* file may be created later by the user or by the Shiny interface, but it is not included as a versioned repository file.

2.3 R PACKAGE REQUIREMENTS

The pipeline uses several R packages for reading files, data processing, normalization, statistics, plotting, and Shiny interface functionality.

The main required packages include:

- shiny
- shinyjs
- bslib
- tidyverse
- readr
- readxl
- openxlsx
- pheatmap
- ggrepel
- stringi
- RColorBrewer
- processx
- shinyFiles
- magick
- pmp
- limma

Some packages are installed from CRAN, while others are installed from Bioconductor. The first package installation may take several minutes depending on the operating system and internet connection.

2.4 INPUT FILE REQUIREMENTS

At minimum, the workflow requires:

1. **Compound Discoverer feature table**

This file contains the metabolomics features, annotation columns, retention time, m/z values, and sample intensity columns.

2. **Experimental metadata file**

This file describes the samples used in the experiment. It should contain variables such as sample name, group, sex, model, sample type, and weight when applicable.

Additional optional files may be used depending on the selected workflow:

3. **Injection order file**

Recommended when using QC-based normalization methods, such as QC-LOESS or QC-RSC. This file provides the analytical order in which samples were injected.

4. **Reference or comparison file**

Optional file used for reference-based duplicate handling or annotation comparison.

2.5 SUPPORTED INPUT FORMATS

The application supports common spreadsheet and text-based formats, including:

- .xlsx
- .xls
- .csv
- .tsv

2.6 RECOMMENDED OPERATING SYSTEMS

CD-MetaboRefine can be used on:

- Windows 10 or newer;
- Linux.
- macOS;

The application was primarily developed and tested in an RStudio-based workflow. For beginners, using RStudio Desktop is strongly recommended.

IMPORTANT NOTES BEFORE STARTING:

- Before running the application, check that:
- R is installed before RStudio;
- the project folder has been fully extracted;
- the .Rproj file is opened from the extracted project folder;
- input files are not open in Excel during pipeline execution;
- sample names in the metadata file match the sample names in the Compound Discoverer table;
- QC samples are clearly identified when QC-based correction is selected;
- the weight column is available if weight normalization is enabled;
- the output folder is in a directory where the user has written permission.

REMEMBER:

Required:

R + RStudio
Project folder
Compound Discoverer table
Metadata file

Recommended:

Internet connection
QC/injection order information
Git for advanced users

3. INSTALL R AND Rstudio

CD-MetaboRefine requires **R** and **RStudio Desktop** to run properly.

R is the programming language used by the pipeline to process the metabolomics data. RStudio Desktop is the recommended environment for opening the project, launching the Shiny application, editing configuration files, and running the script-based workflow.

Although the pipeline can be executed directly from the command line, RStudio is strongly recommended for beginners because it provides a visual interface for scripts, files, plots, console messages, and project organization.

The recommended installation order is:

1. Install **R**.
2. Install **RStudio Desktop**.
3. Open the project using the .Rproj file.

3.1 HOW TO INSTALL R

1. Open the official CRAN website: <https://cran.r-project.org>
2. Select your operating system:
 - Windows;
 - macOS;
 - Linux.
3. Download the R installer recommended for your operating system (Figure 1).
4. Run the installer and keep the default installation options.
5. Wait until the installation finishes successfully.
6. After installation, open R or RStudio and confirm that R is available.

This tutorial was prepared and tested using R version 4.5.3. Users are encouraged to install a recent stable version of R from CRAN. If a different R version is used, package installation and pipeline execution should be checked carefully.

IMPORTANT:

Install R before installing RStudio Desktop. RStudio needs an installed R version to work correctly.

Download and Install R

Precompiled binary distributions of the base system and contributed packages, **Windows and Mac** users most likely want one of these versions of R:

- [Download R for Linux \(Debian, Fedora/Redhat, Ubuntu\)](#)
- [Download R for macOS](#)
- [Download R for Windows](#)

R is part of many Linux distributions, you should check with your Linux package management system in addition to the link above.

Source Code for all Platforms

Windows and Mac users most likely want to download the precompiled binaries listed in the upper box, not the source code. The sources have to be compiled before you can use them. If you do not know what this means, you probably do not want to do it!

- The latest release (2026-06-24, Happy Hop) [R-4.6.1.tar.gz](#), read [what's new](#) in the latest version.
- The CRAN directory [src/base-prerelease](#) contains R alpha, beta, and rc releases as daily snapshots in time periods before a planned release.
- Between releases, the same directory [src/base-prerelease](#) contains snapshots of current patched and development versions. Please read about [new features and bug fixes](#) before filing corresponding feature requests or bug reports.
- Alternatively, daily snapshots are [available here](#).
- Source code of older versions of R is [available here](#).
- Contributed extension [packages](#).

Questions About R

- If you have questions about R like how to download and install the software, or what the license terms are, please read our [answers to frequently asked questions](#) before you send an email.

Supporting CRAN

- CRAN operations, most importantly hosting, checking, distributing, and archiving of R add-on packages for various platforms, crucially rely on technical, emotional, and financial support by the R community.

Please consider making [financial contributions](#) to the R Foundation for Statistical Computing.

Figure 1. CRAN download page showing the Windows installer for R.

TIP:

Use the latest stable R release whenever possible but keep a record of the R version used in each analysis.

3.2 HOW TO INSTALL RSTUDIO DESKTOP

1. Open the Posit download page:
<https://posit.co/download/rstudio-desktop/>
2. Download the free version of **RStudio Desktop** compatible with your operating system (Figure 2).
3. Run the installer using the default options.
4. Open RStudio Desktop after installation.
5. Confirm that RStudio automatically detects the installed R version.

If RStudio does not detect R automatically, close RStudio, check that R was installed correctly, and restart RStudio. On Windows, this issue can sometimes happen when R was installed after RStudio or when the installation path was not detected correctly.

RStudio IDE

Direct download links for RStudio Desktop 2026.05.1, the version this documentation describes:

Platform	Download
Windows	RStudio-2026.05.1-225.exe 
Windows (zip archive)	RStudio-2026.05.1-225.zip 
macOS 13+	RStudio-2026.05.1-225.dmg 
Ubuntu 22 / 24, Debian 12 / 13	rstudio-2026.05.1-225-amd64.deb 
Ubuntu / Debian (tar.gz)	rstudio-2026.05.1-225-amd64-debian.tar.gz 
RHEL 9 / 10, Fedora	rstudio-2026.05.1-225-x86_64.rpm 
RHEL / Fedora (tar.gz)	rstudio-2026.05.1-225-x86_64-fedora.tar.gz 

Figure 2. Posit download page used to download RStudio Desktop.

3.3 RECOMMENDED CHECK AFTER INSTALLATION

After installing R and RStudio, open RStudio and check the Console panel. The Console should show the R version currently being used.

This step is useful because it confirms that:

- R is installed;
- RStudio can detect R;
- the correct R version is being used;
- the project can be opened and executed from RStudio.

If RStudio opens correctly and the Console is available, you can continue to the next step and download the CD-MetaboRefine project.

4. DOWNLOAD THE PROJECT

To use **CD-MetaboRefine**, the project files must first be downloaded to your computer.

There are two possible ways to download the project:

1. **Download the repository as a ZIP file**

Recommended for beginners and users who do not need Git.

2. **Clone the repository using Git**

Recommended for advanced users, developers, or users who want to keep the local project synchronized with future updates.

For most users, the ZIP download method is the simplest and safest option.

4.1 OPTION A — DOWNLOAD AS ZIP

This method does not require Git or command-line tools.

1. Open the GitHub repository page in your browser:
<https://github.com/ianca-kpa/compound-discoverer-metabolomics-pipeline-in-R>
2. Click the green **Code** button.
3. Select **Download ZIP** (Figure 3).

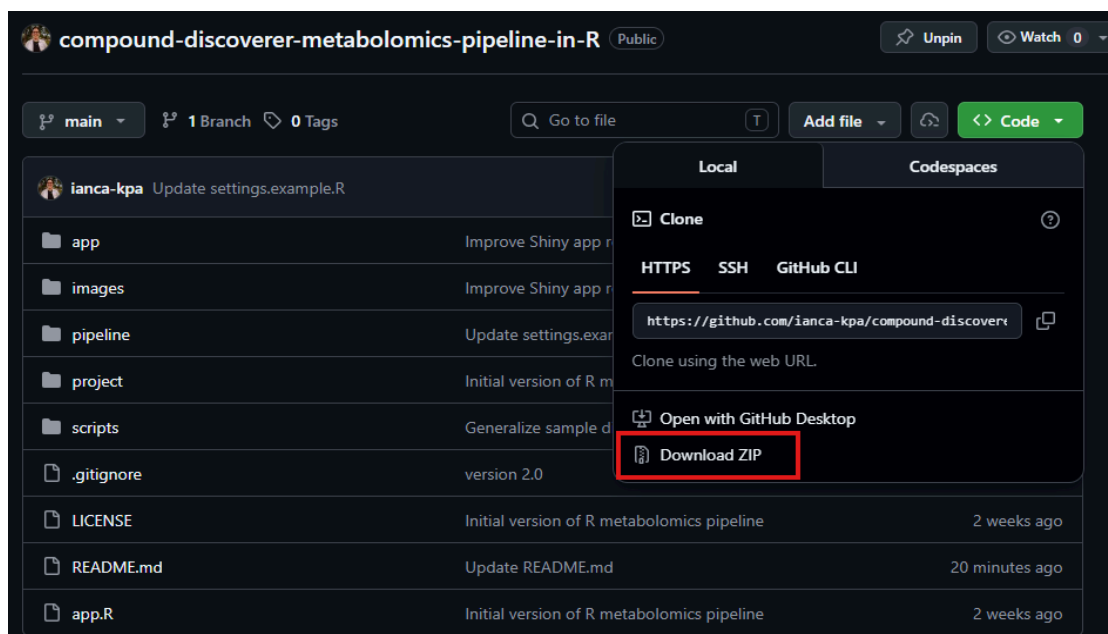


Figure 3. Downloading CD-MetaboRefine from GitHub using Code → Download ZIP.

4. Wait for the download to finish.
5. Locate the downloaded ZIP file on your computer.
6. Extract the ZIP file:
 - **Windows:** right-click the ZIP file and select **Extract All**;
 - **macOS:** double-click the ZIP file;

- **Linux:** use your preferred archive extraction tool.
- 7. Move the extracted project folder to a permanent and easy-to-find location.
- 8. Open the extracted project folder and confirm that it contains files and folders such as:
 - *app.R*;
 - *app/*;
 - *pipeline/*;
 - *images/*;
 - *project/*;
 - *metabolomics-pipeline_git.Rproj*.

IMPORTANT:

Do not run CD-MetaboRefine directly from the compressed ZIP archive. The project must be extracted before it is opened in RStudio.

TIP - RECOMMENDED FOLDER LOCATION:

Choose a simple folder path, preferably without special characters, accents, or very long directory names. This reduces the risk of file path problems during pipeline execution.

Examples of recommended locations:

- *Documents/CD-MetaboRefine/*
- *Desktop/CD-MetaboRefine/*
- *C:/Users/username/Documents/CD-MetaboRefine/*

Avoid running the project from temporary folders, email attachments, cloud preview folders, or directly inside the ZIP archive.

4.2 OPTION B — CLONE WITH GIT

This method is recommended for users who are familiar with Git and want to keep the project synchronized with the GitHub repository.

1. Open a terminal:
 - **Windows:** PowerShell;
 - **macOS:** Terminal;
 - **Linux:** Terminal.
2. Navigate to the folder where you want to store the project.

3. Run the following command:

</> Powershell

```
git clone  
https://github.com/ianca-kpa/compound-discoverer-metabolomics-pipeline-in-R.git
```

4. Enter the downloaded project folder:

</> Powershell

```
cd compound-discoverer-metabolomics-pipeline-in-R
```

5. Open the project in RStudio using the .Rproj file.

This method is useful if you plan to update the project using Git commands or contribute to the repository.

4.3 AFTER DOWNLOADING THE PROJECT

After downloading or cloning the repository, check that the main project structure is present (Figure 4).

The folder should contain:

- *app.R*, used to launch the Shiny application;
- *app/*, containing the CD-MetaboRefine interface files;
- *pipeline/*, containing the analysis scripts;
- *pipeline/config/*, containing *settings.example.R*, the configuration template used to create local analysis settings;
- *images/*, containing documentation or interface images;
- *project/*, containing project-related resources;
- *metabolomics-pipeline_git.Rproj*, used to open the project correctly in RStudio.

If these files and folders are visible, the project was downloaded correctly and is ready to be opened in RStudio.

IMPORTANT:

- Do NOT run the project directly from the ZIP archive.
- Always extract the files before opening the project in RStudio.
- Keep the full folder structure unchanged after extraction.

```

-- app.R
-- app/
|  -- global.R
|  -- server.R
|  -- ui.R
|  -- assets/
|  `-- styles.css
|  `-- helpers/
|      |  -- helpers.R
|      |  -- metadata_helpers.R
|      |  -- server_data_overview_helpers.R
|      |  -- server_gallery_helpers.R
|      |  -- server_input_helpers.R
|      |  -- server_pipeline_helpers.R
|      |  -- server_settings_helpers.R
|      |  `-- server_util_helpers.R
-- pipeline/
|  -- run_pipeline.R
|  -- config/
|  `-- settings.example.R
|  `-- R/
|      |  -- 00_packages.R
|      |  -- 01_validation.R
|      |  -- 02_comparisons.R
|      |  -- 03_helpers_io_log.R
|      |  -- 03a_logging_helpers.R ... 03f_runtime_console_helpers.R
|      |  -- 04_metadata.R
|      |  -- 05_features_assay.R
|      |  -- 06_normalization_filters.R
|      |  -- 06a_normalization_core.R ... 06c_filter_helpers.R
|      |  -- 07_duplicates.R
|      |  -- 08_exports.R
|      |  -- 09_pca.R
|      |  -- 10_stats_volcano.R
|      |  -- 10a_stats_core.R ... 10d_stats_exports.R
|      |  -- 11_heatmaps.R
|      |  `-- 12_main_pipeline.R
-- scripts/
-- images/
`-- project/

```

Figure 4. Repository structure of the metabolomics pipeline.

5. OPEN THE PROJECT IN RStudio

After downloading and extracting the repository, the project should be opened correctly in RStudio.

Opening the project correctly is important because CD-MetaboRefine uses relative paths to locate the application files, pipeline scripts, configuration files, input data, images, and output folders.

5.1 OPEN THE PROJECT USING THE .RPROJ FILE

1. Open **RStudio Desktop**.
2. In the top menu, click: **File** → **Open Project...**
3. Navigate to the extracted CD-MetaboRefine project folder.
4. Select the project file:
CD-MetaboRefine.Rproj
5. Click **Open**.

After opening the project, the **Files** panel in RStudio should show the main project folders, including:

- app/
- pipeline/
- scripts/
- images/

This confirms that RStudio is working from the correct project directory.

5.2 WHY THIS STEP IS IMPORTANT

Opening the project using the .Rproj file ensures that:

- relative paths work correctly;
- the Shiny application can locate the files inside app/;
- the pipeline can locate scripts inside pipeline/R/;
- configuration files are read from the correct location;
- output files are written to the correct output directory;
- the analysis remains organized and reproducible.

If the project is not opened correctly, the application may fail to find files, scripts, images, configuration files, or output folders.

5.3 IF NO .Rproj FILE IS AVAILABLE

If the repository does not contain a .Rproj file, the project can still be opened from RStudio by creating a new project from the existing folder.

1. Open **RStudio Desktop**.

2. Click: **File** → **New Project...**
3. Select: **Existing Directory**
4. Browse and select the extracted CD-MetaboRefine folder.
5. Click: **Create Project**

RStudio will create a new .Rproj file inside the selected folder.

IMPORTANT:

- Do not open individual .R files directly before opening the project folder.
- Do not run the project from inside the compressed ZIP archive.
- Keep the complete folder structure unchanged.
- Avoid moving files manually between folders.
- Avoid using very long folder paths or paths with special characters.
- Make sure the project root contains the app/ and pipeline/ folders.

Once the project is open in RStudio, you can continue to the next step and install the required R packages.

6. INSTALL REQUIRED PACKAGES

Before launching CD-MetaboRefine for the first time, the required R packages must be installed.

The application and the pipeline use different packages for the Shiny interface, file reading, data processing, normalization, statistical analysis, plotting, and export generation.

6.1 INSTALL THE STARTUP PACKAGES

Some packages are required immediately when the Shiny application starts. These packages should be installed manually before launching the app for the first time.

Open the RStudio Console and run:

```
</> R
```

```
install.packages(c("shiny", "shinyjs", "bslib"))
```

These packages are required because the application interface depends on them during startup.

6.2 AUTOMATIC INSTALLATION OF PIPELINE DEPENDENCIES

After the startup packages are available, CD-MetaboRefine can check and install the remaining pipeline dependencies using the package management script: ***pipeline/R/00_packages.R***

This script checks whether the required packages are installed. If some packages are missing, it attempts to install them automatically.

The main CRAN packages used by the pipeline include:

- tidyverse
- readr
- readxl
- openxlsx
- pheatmap
- ggrepel
- stringi
- RColorBrewer
- processx
- shinyFiles
- magick

The pipeline also uses Bioconductor packages, including:

- pmp
- limma

If the Bioconductor packages are missing, the pipeline installs BiocManager when needed and then uses it to install the required Bioconductor dependencies.

6.3 OPTIONAL FULL MANUAL INSTALLATION

If automatic installation does not work, the main CRAN packages can be installed manually from the RStudio Console:

```
</> R
```

```
install.packages(c(
  "shiny", "shinyjs", "bslib", "tidyverse", "readr", "readxl", "openxlsx",
  "pheatmap", "ggrepel", "stringi", "RColorBrewer", "processx",
  "shinyFiles", "magick", "BiocManager"
))
```

Then install the Bioconductor packages:

```
</> R
```

```
BiocManager::install(c("pmp", "limma"), ask = FALSE, update = FALSE)
```

6.4 WHY PACKAGE INSTALLATION MAY TAKE TIME

The first installation may take several minutes, especially on a new computer.

Installation time depends on:

- internet connection speed;
- operating system;
- R version;
- whether packages need to be compiled;
- whether system libraries are already available.

On Linux and macOS, some packages may require additional system libraries or compilation tools. On Windows, installing packages from binary versions is usually simpler.

IMPORTANT:

- Keep a stable internet connection during the first installation.
- Restart RStudio after installing many packages.
- If a package installation fails, read the Console message carefully.
- If the app does not open, first check whether shiny, shinyjs, and bslib are installed.
- If the pipeline fails during execution, inspect the package messages in the Console and in the pipeline log.
- Package installation is usually required only once per R library environment.

7. APPLICATION AND PIPELINE STRUCTURE

CD-MetaboRefine is organized into two main components:

1. **The Shiny application**, which provides the graphical interface used to upload files, configure settings, run the pipeline, inspect logs, and explore results.
2. **The modular R pipeline**, which performs the analytical workflow, including validation, metadata processing, feature table preparation, normalization, filtering, duplicate handling, statistical analysis, visualization, and export generation.

The project follows a modular structure so that the user interface and the analytical workflow remain separated. This makes the project easier to maintain, update, debug, and reuse.

7.1 MAIN PROJECT FOLDERS

The main project folders are described in Table I.

Table I. Main project folders and their purpose.

Folder	Purpose
<i>app/</i>	Contains the Shiny application used to interact with CD-MetaboRefine.
<i>app/assets/</i>	Contains files used to style the application interface, such as CSS.
<i>app/helpers/</i>	Contains helper scripts used by the Shiny server logic.
<i>pipeline/</i>	Contains the R-based metabolomics pipeline.
<i>pipeline/config/</i>	Contains the example and local configuration files.
<i>pipeline/R/</i>	Contains the modular analysis scripts used by the pipeline.
<i>scripts/</i>	Contains auxiliary scripts for diagnostics, reports, or additional analyses.
<i>images/</i>	Contains images used in the README, tutorial, or documentation.

The most important folders for standard users are *app/*, *pipeline/*, and *images/*.

7.2 SHINY APPLICATION FILES

The Shiny application is in the `app/` directory. The main application files are described in Table II.

Table II. Main Shiny application files.

File	Purpose
<i>app/global.R</i>	Loads shared packages, paths, helper functions, and resources used by the application.
<i>app/ui.R</i>	Defines the visible layout of the CD-MetaboRefine interface.
<i>app/server.R</i>	Defines the application logic, including file upload, settings management, pipeline execution, logs, and result display.
<i>app/assets/styles.css</i>	Controls the visual style of the application, including colors, cards, spacing, buttons, and layout.

The application also uses helper scripts located in *app/helpers/*.

7.3 APPLICATION HELPER SCRIPTS

The *app/helpers/* folder contains scripts that support specific parts of the Shiny application (Table III).

Table III. Helper scripts used by the CD-MetaboRefine Shiny application.

File	Purpose
<i>helpers.R</i>	Provides general helper functions used by the application.
<i>metadata_helpers.R</i>	Supports metadata reading, checking, and preparation.
<i>server_data_overview_helpers.R</i>	Supports the Data Overview tab and input inspection.
<i>server_gallery_helpers.R</i>	Supports the Results Gallery tab and result image display.
<i>server_input_helpers.R</i>	Supports input file selection and upload behavior.
<i>server_pipeline_helpers.R</i>	Supports pipeline execution from the Shiny interface.
<i>server_settings_helpers.R</i>	Supports the Settings Builder and configuration options.
<i>server_util_helpers.R</i>	Provides additional utility functions used by the server logic.

This organization keeps *server.R* cleaner by separating large blocks of logic into smaller helper files.

7.4 MAIN APPLICATION TABS

The CD-MetaboRefine interface is organized into main tabs that guide the user through the workflow.

7.4.1 Data Overview

The Data Overview tab displays information about the uploaded files.

It helps the user verify whether the input files were correctly loaded and whether the samples were recognized by the application.

This tab is useful for checking:

- uploaded Compound Discoverer data;
- uploaded metadata;
- sample names;
- detected sample columns;
- metadata alignment;
- possible input inconsistencies.

7.4.2 Settings Builder

The Settings Builder tab allows the user to configure the main analysis options through the graphical interface.

It includes options related to:

- statistical analysis;
- p-value and FDR thresholds;
- fold-change cutoff;
- normalization method;
- weight normalization;
- feature filtering;
- duplicate handling;
- PCA settings;
- volcano plot settings;
- heatmap settings;
- export level.

This tab reduces the need to manually edit configuration files and makes the workflow more accessible to users with limited programming experience.

7.4.3 Pipeline Log

The Pipeline Log tab displays execution messages generated during the run.

It helps the user monitor:

- whether the pipeline started correctly;

- which steps were executed;
- warnings;
- possible errors;
- completion messages.

The log should be checked after every run.

7.4.4 Results Gallery

The Results Gallery tab displays generated result images after the pipeline has finished.

It provides quick access to visual outputs such as:

- PCA plots;
- volcano plots;
- heatmaps;
- normalization comparison plots;
- QC diagnostic plots.

7.5 PIPELINE CONFIGURATION FILES

The pipeline configuration files are in *pipeline/config/*

The repository includes only the example configuration file: *settings.example.R*

This file is the configuration template provided with the project and should be used as a reference for creating local analysis settings.

The *settings.example.R* file should remain unchanged because it serves as the default template.

During local use, a *settings.R* file may be created by the user or generated from the Shiny interface. This local file stores user-specific paths and analysis options for a specific run.

Because *settings.R* may contain local file paths and user-specific settings, it should not be committed to the public repository.

7.6 PIPELINE ANALYSIS MODULES

The main analytical scripts are located in *pipeline/R/*. Each script is responsible for one part of the metabolomics workflow (Table IV).

Table IV. Modular organization of the CD-MetaboRefine pipeline.

Module	Purpose
<i>00_packages.R</i>	Loads, checks, and installs required R packages when needed.
<i>01_validation.R</i>	Validates settings and configuration parameters.
<i>02_comparisons.R</i>	Defines comparison groups used for statistical analyses.
<i>03_helpers_io_log.R</i>	Loads helper functions for input/output operations and logging.
<i>03a_logging_helpers.R</i>	Provides logging helper functions.
<i>03b_io_reference_helpers.R</i>	Provides input/output and reference file helper functions.
<i>03c_text_config_helpers.R</i>	Supports text and configuration handling.
<i>03d_setting_value_helpers.R</i>	Supports setting value validation and management.
<i>03e_summary_path_helpers.R</i>	Supports summary and output path management.
<i>03f_runtime_console_helpers.R</i>	Supports runtime and console messages.
<i>04_metadata.R</i>	Cleans and standardizes metadata tables.
<i>05_features_assay.R</i>	Builds feature tables and assay matrices from Compound Discoverer exports.
<i>06_normalization_filters.R</i>	Coordinates normalization and filtering procedures.
<i>06a_normalization_core.R</i>	Contains core normalization functions.
<i>06b_normalization_plots.R</i>	Generates normalization and QC diagnostic plots.
<i>06c_filter_helpers.R</i>	Provides helper functions for filtering procedures.
<i>07_duplicates.R</i>	Handles duplicate metabolite names and duplicate processing strategies.
<i>08_exports.R</i>	Creates export files for downstream tools such as MetaboAnalyst.
<i>09_pca.R</i>	Generates PCA plots.
<i>10_stats_volcano.R</i>	Coordinates statistical analysis and volcano plot generation.
<i>10a_stats_core.R</i>	Contains core statistical analysis functions.
<i>10b_volcano_plots.R</i>	Generates volcano plots.
<i>10c_stats_driver.R</i>	Coordinates statistical comparisons.
<i>10d_stats_exports.R</i>	Exports statistical result tables.
<i>11_heatmaps.R</i>	Generates heatmap visualizations.
<i>12_main_pipeline.R</i>	Coordinates the complete pipeline execution.

7.7 MAIN EXECUTION SCRIPT

The main script used to execute the pipeline from code is *pipeline/run_pipeline.R*

This script loads the configuration files and pipeline modules, then starts the complete analysis workflow.

Users who run the workflow from the Shiny application do not usually need to edit this file.

7.8 AUXILIARY SCRIPTS

The scripts/ folder contains additional scripts that may be useful for diagnostics, reports, or development. Examples are presented in Table V.

Table V. Auxiliary scripts included in the project

File	Purpose
<i>diagnose_sample.R</i>	Helps diagnose sample-related issues.
<i>generate_qc_loess_weight_plot.R</i>	Generates QC-LOESS and weight-related diagnostic plots.
<i>generate_sample_report.R</i>	Generates sample-level reports.
<i>sample_report.Rmd</i>	R Markdown template for sample reports.
<i>README.md</i>	Describes the purpose of the auxiliary scripts folder.

These scripts are not required for a standard Shiny app run, but they may be useful for advanced users or debugging.

7.9 HOW THE COMPONENTS WORK TOGETHER

The workflow operates as follows:

1. The user opens the project on RStudio.
2. The user launches CD-MetaboRefine.
3. *app/global.R* loads shared resources and helper scripts.
4. *app/ui.R* builds the graphical interface.
5. *app/server.R* controls user interactions, upload files, settings, logs, and results.
6. Helper scripts in *app/helpers/* support specific parts of the application.
7. The selected settings are passed to the pipeline.
8. The scripts in *pipeline/R/* process the data step by step.
9. The pipeline writes tables, plots, logs, and audit files to the output directory.
10. The Shiny app displays selected results in the Results Gallery.

This separation between the Shiny interface and the pipeline scripts makes CD-MetaboRefine easier to use interactively while still supporting reproducible script-based analyses.

8. PREPARE THE INPUT FILES

Before running CD-MetaboRefine, the input files must be prepared carefully. Most execution problems occur when sample names, metadata labels, QC information, or file formats are inconsistent.

At minimum, the workflow requires:

1. a **Compound Discoverer feature table**;
2. an **experimental metadata file**.

Depending on the selected analysis options, additional files may also be used:

3. an **injection order file**, recommended for QC-based normalization;
4. a **reference or comparison file**, optional for reference-based annotation or duplicate handling.

Table VI summarizes the input files used by CD-MetaboRefine.

Table VI. Input files used by CD-MetaboRefine.

File	Required?	Purpose
Compound Discoverer feature table	Yes	Contains metabolomics features, annotation columns, retention time, m/z values, and sample intensity columns.
Metadata file	Yes	Describes the biological and technical information associated with each sample.
Injection order file	Recommended for QC-based correction	Provides the analytical run order used for QC-LOESS or QC-RSC normalization.
Reference/comparison file	Optional	Used for reference-based duplicate handling or annotation comparison.

8.1 COMPOUND DISCOVERER FEATURE TABLE

The Compound Discoverer feature table is the main metabolomics input file. It contains the detected features, annotation information, and abundance values measured for each sample.

This file usually contains:

- metabolite or feature names;
- molecular formula, when available;
- m/z values;
- retention time;
- annotation-related columns;
- sample intensity columns, usually identified as Area columns;
- quality-control information, when exported by Compound Discoverer.

Supported formats include:

- .xlsx;
- .xls;
- .csv;
- .tsv.

IMPORTANT RECOMMENDATIONS:

- Do not rename sample columns unless necessary.
- Keep the original Compound Discoverer export structure as much as possible.
- Make sure that sample intensity or Area columns are present.
- Avoid manually deleting important annotation columns.
- Avoid merged cells, empty header rows, or modified spreadsheet structures.
- Keep a backup copy of the original Compound Discoverer export before editing.

8.2 METADATA FILE

The metadata file describes the samples included in the analysis. It connects each sample in the Compound Discoverer table to biological and experimental information.

Typical metadata columns are described in Table VII.

Table VII. Recommended metadata columns.

Column	Description	Required?
<i>sample</i>	Sample name matching the Compound Discoverer table.	Yes
<i>group</i>	Biological or experimental group, such as WT or TG.	Recommended
<i>sex</i>	Sex of the animal or donor, such as F or M.	Optional, depending on design
<i>model</i>	Experimental model, such as AETA-m or APPdeIETA.	Optional, depending on design
<i>type</i>	Sample type, such as biological sample or QC sample.	Recommended when QC samples are present
<i>weight_mg</i>	Sample or tissue weight used for weight normalization.	Required if weight normalization is enabled

The most important rule is that the values in the sample column must match the sample names detected in the Compound Discoverer table. Example (Table VIII):

Table VIII. Example metadata table.

sample	type	weight_mg	sex	group	model
S01	Sample	18.5	F	WT	model-Y

S02	Sample	20.1	M	TG	model-X
QC_01	QC	NA	NA	NA	NA

IMPORTANT RECOMMENDATIONS:

- Use one row per sample.
- Avoid duplicated sample names.
- Avoid empty rows.
- Avoid merged cells.
- Keep group names consistent.
- Use the same spelling across the whole file.
- Do not mix labels such as WT, Wt, wildtype, and Wild Type unless they are intentionally different.
- Clearly identify QC samples when QC-based correction will be used.

8.3 INJECTION ORDER FILE

The injection order file provides the analytical order in which the samples were injected into the LC-MS/MS system.

This information is important for QC-based normalization methods, such as QC-LOESS or QC-RSC, because these methods use QC samples distributed across the analytical sequence to estimate and correct signal drift over time.

The injection order can be created manually, but it can also be obtained from Compound Discoverer. In Compound Discoverer, the Input Files tab contains information about the raw files included in the study, such as file name, creation date, batch, sample type, model, sex, and group. This table can be exported to Excel and used to prepare the injection order file.

To obtain this information from Compound Discoverer:

1. Open the study in Compound Discoverer.
2. Go to the **Input Files** tab (Figure 5).
3. Check that the raw files are listed in the correct analytical order.
4. Export the table to Excel.
5. Use the exported file to create or verify the injection order table.

Compounds																	
Compounds		Compounds per File		Features per File		mzCloud 1 Results		ChemSpider Results		Input Files		Study Information		Statistical Methods		Metabolika Pathways	
	Study File ID	File Name	Creation Date	Batch	RT Range [min]	Instrument Name	Software Revision	Ref. File ID	Sample Type	Model	Sex	Group	Max. Mass [Da]				
1	F1	QC_ddMS2_Pos11.raw	11/18/2025 9:45 PM	-1	0.00 - 22.00	Q Exactive Plus Orbitrap	2.13-213012/2.13.0.3162	F28	Identification Only	n/a	n/a	n/a	6000.00000				
2	F2	QC_MS_Pos11.raw	11/18/2025 9:17 PM	0	0.01 - 22.01	Q Exactive Plus Orbitrap	2.13-213012/2.13.0.3162	F28	Quality Control	n/a	n/a	n/a	900.00000				
3	F3	H8_ddMS2_Pos1.raw	11/18/2025 8:20 PM	-1	0.00 - 22.00	Q Exactive Plus Orbitrap	2.13-213012/2.13.0.3162	F72	Identification Only	APPdELTA	Male	TG	6000.00000				
4	F4	H8_MS_Pos.raw	11/18/2025 7:52 PM	0	0.01 - 22.01	Q Exactive Plus Orbitrap	2.13-213012/2.13.0.3162	F72	Sample	APPdELTA	Male	TG	900.00000				
5	F5	H7_ddMS2_Pos1.raw	11/18/2025 6:55 PM	-1	0.00 - 22.00	Q Exactive Plus Orbitrap	2.13-213012/2.13.0.3162	F72	Identification Only	APPdELTA	Male	TG	6000.00000				
6	F6	H7_MS_Pos.raw	11/18/2025 6:27 PM	0	0.01 - 22.00	Q Exactive Plus Orbitrap	2.13-213012/2.13.0.3162	F72	Sample	APPdELTA	Male	TG	900.00000				
7	F7	G8_ddMS2_Pos1.raw	11/18/2025 5:30 PM	-1	0.00 - 22.00	Q Exactive Plus Orbitrap	2.13-213012/2.13.0.3162	F120	Identification Only	APPdELTA	Male	WT	6000.00000				
8	F8	G8_MS_Pos.raw	11/18/2025 5:02 PM	0	0.01 - 22.00	Q Exactive Plus Orbitrap	2.13-213012/2.13.0.3162	F120	Sample	APPdELTA	Male	WT	900.00000				
9	F9	G7_ddMS2_Pos1.raw	11/18/2025 4:05 PM	-1	0.00 - 22.00	Q Exactive Plus Orbitrap	2.13-213012/2.13.0.3162	F120	Identification Only	APPdELTA	Male	WT	6000.00000				
10	F10	G7_MS_Pos.raw	11/18/2025 3:37 PM	0	0.01 - 22.00	Q Exactive Plus Orbitrap	2.13-213012/2.13.0.3162	F120	Sample	APPdELTA	Male	WT	900.00000				
11	F11	F8_ddMS2_Pos1.raw	11/18/2025 2:41 PM	-1	0.00 - 22.00	Q Exactive Plus Orbitrap	2.13-213012/2.13.0.3162	F32	Identification Only	APPdELTA	Female	TG	6000.00000				
12	F12	F8_MS_Pos.raw	11/18/2025 2:12 PM	0	0.01 - 22.01	Q Exactive Plus Orbitrap	2.13-213012/2.13.0.3162	F32	Sample	APPdELTA	Female	TG	900.00000				
13	F13	F7_ddMS2_Pos1.raw	11/18/2025 1:16 PM	-1	0.00 - 22.00	Q Exactive Plus Orbitrap	2.13-213012/2.13.0.3162	F32	Identification Only	APPdELTA	Female	TG	6000.00000				
14	F14	F7_MS_Pos.raw	11/18/2025 12:47 PM	0	0.01 - 22.00	Q Exactive Plus Orbitrap	2.13-213012/2.13.0.3162	F32	Sample	APPdELTA	Female	TG	900.00000				
15	F15	QC_ddMS2_Pos10.raw	11/18/2025 11:51 AM	-1	0.00 - 22.00	Q Exactive Plus Orbitrap	2.13-213012/2.13.0.3162	F28	Identification Only	n/a	n/a	n/a	6000.00000				
16	F16	QC_MS_Pos10.raw	11/18/2025 11:23 AM	0	0.01 - 22.00	Q Exactive Plus Orbitrap	2.13-213012/2.13.0.3162	F28	Quality Control	n/a	n/a	n/a	900.00000				
17	F17	E8_ddMS2_Pos1.raw	11/18/2025 10:26 AM	-1	0.00 - 22.00	Q Exactive Plus Orbitrap	2.13-213012/2.13.0.3162	F18	Identification Only	APPdELTA	Female	WT	6000.00000				
18	F18	E8_MS_Pos.raw	11/18/2025 9:58 AM	0	0.01 - 22.00	Q Exactive Plus Orbitrap	2.13-213012/2.13.0.3162	F32	Sample	APPdELTA	Female	WT	900.00000				
19	F19	E7_ddMS2_Pos1.raw	11/18/2025 9:01 AM	-1	0.00 - 22.00	Q Exactive Plus Orbitrap	2.13-213012/2.13.0.3162	F18	Identification Only	APPdELTA	Female	WT	6000.00000				
20	F20	E7_MS_Pos.raw	11/18/2025 8:33 AM	0	0.01 - 22.00	Q Exactive Plus Orbitrap	2.13-213012/2.13.0.3162	F18	Sample	APPdELTA	Female	WT	900.00000				
21	F21	D8_ddMS2_Pos1.raw	11/18/2025 7:36 AM	-1	0.00 - 22.00	Q Exactive Plus Orbitrap	2.13-213012/2.13.0.3162	F80	Identification Only	AETA-m	Male	TG	6000.00000				
22	F22	D8_MS_Pos.raw	11/18/2025 7:08 AM	0	0.01 - 22.00	Q Exactive Plus Orbitrap	2.13-213012/2.13.0.3162	F80	Sample	AETA-m	Male	TG	900.00000				
23	F23	D7_ddMS2_Pos1.raw	11/18/2025 6:11 AM	-1	0.00 - 22.00	Q Exactive Plus Orbitrap	2.13-213012/2.13.0.3162	F80	Identification Only	AETA-m	Male	TG	6000.00000				
24	F24	D7_MS_Pos.raw	11/18/2025 5:43 AM	0	0.01 - 22.00	Q Exactive Plus Orbitrap	2.13-213012/2.13.0.3162	F80	Sample	AETA-m	Male	TG	900.00000				
25	F25	H6_ddMS2_Pos1.raw	11/18/2025 4:46 AM	-1	0.00 - 22.00	Q Exactive Plus Orbitrap	2.13-213012/2.13.0.3162	F72	Identification Only	APPdELTA	Male	TG	6000.00000				
26	F26	H6_MS_Pos.raw	11/18/2025 4:18 AM	0	0.01 - 22.00	Q Exactive Plus Orbitrap	2.13-213012/2.13.0.3162	F72	Sample	APPdELTA	Male	TG	900.00000				
27	F27	QC_ddMS2_Pos9.raw	11/18/2025 3:22 AM	-1	0.00 - 22.00	Q Exactive Plus Orbitrap	2.13-213012/2.13.0.3162	F28	Identification Only	n/a	n/a	n/a	6000.00000				
28	F28	QC_MS_Pos9.raw	11/18/2025 2:53 AM	0	0.01 - 22.00	Q Exactive Plus Orbitrap	2.13-213012/2.13.0.3162	F32	Quality Control	n/a	n/a	n/a	900.00000				
29	F29	G6_ddMS2_Pos1.raw	11/18/2025 1:57 AM	-1	0.00 - 22.00	Q Exactive Plus Orbitrap	2.13-213012/2.13.0.3162	F120	Identification Only	APPdELTA	Male	WT	6000.00000				
30	F30	G6_MS_Pos.raw	11/18/2025 1:28 AM	0	0.01 - 22.00	Q Exactive Plus Orbitrap	2.13-213012/2.13.0.3162	F120	Sample	APPdELTA	Male	WT	900.00000				
31	F31	F6_ddMS2_Pos1.raw	11/18/2025 12:32 AM	-1	0.00 - 22.00	Q Exactive Plus Orbitrap	2.13-213012/2.13.0.3162	F32	Identification Only	APPdELTA	Female	TG	6000.00000				
32	F32	F6_MS_Pos.raw	11/18/2025 12:04 AM	0	0.01 - 22.00	Q Exactive Plus Orbitrap	2.13-213012/2.13.0.3162	F32	Sample	APPdELTA	Female	TG	900.00000				
33	F33	E6_ddMS2_Pos1.raw	11/17/2025 11:07 PM	-1	0.00 - 22.00	Q Exactive Plus Orbitrap	2.13-213012/2.13.0.3162	F18	Identification Only	APPdELTA	Female	WT	6000.00000				
34	F34	E6_MS_Pos.raw	11/17/2025 10:39 PM	0	0.01 - 22.00	Q Exactive Plus Orbitrap	2.13-213012/2.13.0.3162	F18	Sample	APPdELTA	Female	WT	900.00000				
35	F35	D6_ddMS2_Pos1.raw	11/17/2025 9:42 PM	-1	0.00 - 22.00	Q Exactive Plus Orbitrap	2.13-213012/2.13.0.3162	F80	Identification Only	AETA-m	Male	TG	6000.00000				
36	F36	D6_MS_Pos.raw	11/17/2025 9:14 PM	0	0.01 - 22.00	Q Exactive Plus Orbitrap	2.13-213012/2.13.0.3162	F80	Sample	AETA-m	Male	TG	900.00000				

Figure 5. Compound Discoverer Input Files tab showing raw files, sample type, batch information, and experimental metadata columns that can be exported to Excel and used to prepare the injection order file.

A simple injection order file should contain at least the following columns (Table IX):

Table IX. Example injection order file.

sample	injection_order	type
QC_01	1	QC
S01	2	BIO
S02	3	BIO
QC_02	4	QC

The sample column must match the sample names used in the metadata file and in the Compound Discoverer feature table.

If no injection order file is provided, some workflows may still run, but QC-based drift correction may be unavailable, limited, or less reliable.

IMPORTANT:

- Use the real analytical injection order whenever possible.
- Confirm that the exported Compound Discoverer table reflects the true injection sequence.
- Use numeric injection order values.
- Make sure QC samples are correctly labeled.
- Do not alphabetically sort the samples if this changes to the original analytical order.
- Do not use only biological group order as a substitute for injection order.
- Keep QC samples in their original position in the sequence.

8.4 REFERENCE OR COMPARISON FILE

The reference or comparison file is optional. It can be used when the user wants to compare annotations against a predefined reference list or support reference-based duplicate handling.

This file is not required for a standard pipeline run.

Use this file only when:

- a reference annotation list is available;
- duplicate handling should consider an external reference;
- a comparison between exported annotations and reference annotations is needed.

If this option is not part of the analysis, the field can be left empty.

8.5 GOOD PRACTICES BEFORE RUNNING THE PIPELINE

Before running CD-MetaboRefine, check the input files carefully.

RECOMMENDED CHECKS:

- Confirm that all files are saved and closed before running the pipeline.
- Confirm that the Compound Discoverer table contains sample intensity columns.
- Confirm that sample names match between the feature table, metadata file, and injection order file.
- Confirm that group labels are consistent.
- Confirm that QC samples are clearly identified when QC-based normalization is selected.
- Confirm that the weight column is available if weight normalization is enabled.
- Confirm that there are no duplicated sample names.
- Confirm that there are no merged cells in spreadsheet files.
- Confirm that the selected files are not corrupted or empty.
- Prefer simple file names without special characters.

COMMON INPUT PROBLEMS:

- mismatched sample names;
- missing Area or intensity columns;
- duplicated sample names;
- inconsistent group labels;
- missing metadata columns;
- missing weight values when weight normalization is enabled;
- missing QC labels when QC-based normalization is selected;
- incorrect injection order;
- spreadsheet files with merged cells;
- files left open in Excel during execution;
- file paths that are too long or contain special characters.

9. LAUNCH THE APPLICATION AND RUN THE PIPELINE

CD-MetaboRefine includes an interactive Shiny application that allows users to run the metabolomics workflow through a graphical interface.

The application can be used to:

- upload input files;
- select metadata groups;
- configure the output directory;
- review and save analysis settings;
- run the pipeline;
- inspect the execution log;
- browse generated figures and result summaries.

This execution mode is recommended for users who prefer a visual workflow instead of manually editing R scripts.

9.1 STEP 1 — LAUNCH CD-METABOREFINE

After opening the project in RStudio, launch the application from RStudio Console.

Run:

```
</> R
```

```
shiny::runApp("app")
```

The CD-MetaboRefine interface should open in a new window or in the *RStudio Viewer* pane (Figure 6).

The top of the interface displays the application name: CD-MetaboRefine;

and the application description: An R/Shiny pipeline for quality control, normalization, statistical analysis, and visualization of Compound Discoverer metabolomics data.

If the interface opens correctly, the status area should show that the application is ready.

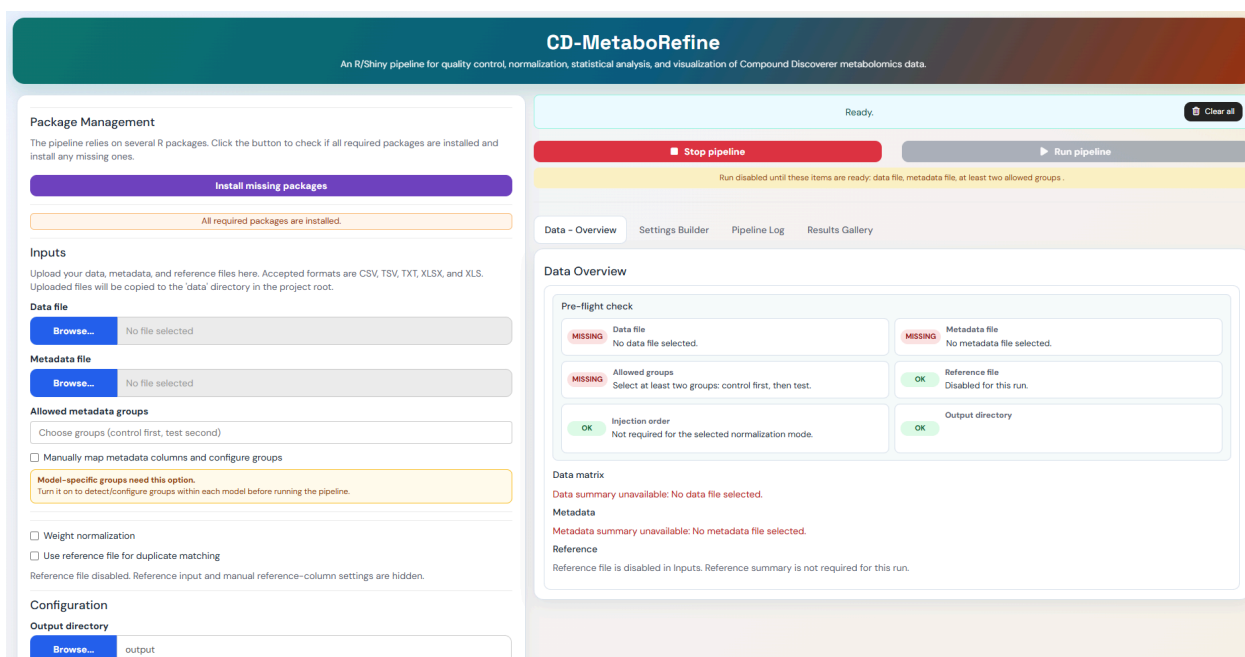


Figure 6. CD-MetaboRefine main interface after launching the Shiny application.

9.2 STEP 2 — CHECK PACKAGE AVAILABILITY

The left panel of the application contains a Package Management section. If required packages are missing, click: **Install missing packages** button.

After installation, the application should display a message indicating that all required packages are installed.

If package installation fails, check the RStudio Console for error messages and confirm that the internet connection is active.

9.3 STEP 3 — UPLOAD THE INPUT FILES

Use the Inputs section to upload the required files (Figure 7).

At minimum, upload:

1. **Data file** — the Compound Discoverer feature table.
2. **Metadata file** — the experimental metadata file.

Optional files or options may include:

3. **Reference file** — used only if reference-based duplicate matching is enabled.
4. **Manual metadata mapping** — used when the application cannot automatically detect the required metadata columns.

After selecting files, the uploaded files are copied to the *data/* directory in the project root.

IMPORTANT CHECKS BEFORE CONTINUING:

1. confirm that the correct data file was selected;
2. confirm that the correct metadata file was selected;
3. confirm that sample names match between the data and metadata files;
4. confirm that QC samples are correctly identified when QC-based normalization will be used;
5. confirm that the reference file is only enabled when needed.

The screenshot displays the 'Inputs' section of the CD-MetaboRefine web interface. It includes instructions for uploading data, metadata, and reference files in CSV, TSV, TXT, XLSX, and XLS formats. Below the instructions are two file selection fields: 'Data file' and 'Metadata file', each with a 'Browse...' button and a 'No file selected' status. The 'Allowed metadata groups' section features a dropdown menu with the text 'Choose groups (control first, test second)' and an unchecked checkbox for 'Manually map metadata columns and configure groups'. A yellow warning box states: 'Model-specific groups need this option. Turn it on to detect/configure groups within each model before running the pipeline.' Below this are two more unchecked checkboxes: 'Weight normalization' and 'Use reference file for duplicate matching'. A note indicates 'Reference file disabled. Reference input and manual reference-column settings are hidden.' The 'Configuration' section includes an 'Output directory' field with a 'Browse...' button and the text 'output'.

Figure 7. Input file selection and output directory configuration in CD-MetaboRefine.

9.4 STEP 4 — SELECT METADATA GROUPS

The Allowed metadata groups field allows the user to choose which groups will be used in the analysis.

When selecting groups, choose the control group first and the test group second. For example: **WT, TG**.

In this case, WT is treated as the control group and TG as the test group. This order is important because it affects the direction of fold-change calculations.

For a TG versus WT comparison, the expected interpretation is:

$$\log_2\text{FC} = \log_2(\text{TG} / \text{WT})$$

Positive $\log_2\text{FC}$ values indicate higher abundance in TG compared with WT, while negative values indicate lower abundance in TG compared with WT.

9.5 STEP 5 — CONFIGURE THE OUTPUT DIRECTORY

In the *Configuration* section, select the output directory where the generated results will be saved.

By default, the application may use *output*

This means that result files will be saved inside an *output/* folder in the project directory.

Before running the pipeline, confirm that:

- the output directory is correct;
- the directory is in a place where the user has written permission;
- the directory path is not too long;
- the output folder is not open in another program that could block file writing.

9.6 STEP 6 — REVIEW THE SETTINGS BUILDER

Open the **Settings Builder** tab to review the main analysis options. The Settings Builder is divided into three main sections:

1. Run setup
2. Visual outputs
3. Exports

- **Run setup**

The **Run setup** section controls the main analysis parameters (Figure 8), including:

- number of groups to compare;
- pairwise test;
- pairwise design;
- significance metric;
- p-value cutoff;
- FDR cutoff;
- FC cutoff;
- preprocessing scenario;
- QC/normalization audit;
- RSD filter;
- RSD threshold;
- low-variance filter;
- IQR fraction;
- known feature filtering;
- duplicate handling.

Data - Overview **Settings Builder** Pipeline Log Results Gallery

Settings form Variable guide

Use the form below to configure key variables with standardized input fields. The variable guide is in the other tab.

Settings form Current values are applied when running the pipeline. [Save config/settings.R from form](#)

Run setup Visual outputs Exports

Statistical analysis
 Multi-group analysis: exploratory and complementary. The five primary pairwise comparisons remain enabled. The global test has no directional FC or volcano plot; select only biologically interpretable follow-up pairs.

How many groups will be compared? Pairwise test
 Primary pairwise comparisons student

Pairwise design Significance metric
 Unpaired FDR_and_p_value

Statistics thresholds
 P-value cutoff FDR cutoff FC cutoff (log2)
 0,05 0,05 0

Normalization
 Preprocessing scenario Run QC/normalization audit
 None FALSE

Feature filters
 RSD filter RSD threshold Low-variance filter
 none 20 none

IQR fraction Use only known features Duplicate handling
 0,2 TRUE collapse_best_qc_rsd

Figure 8. Settings Builder run setup options, including statistical analysis, normalization, filtering, and duplicate handling.

- **Visual outputs**

The **Visual outputs** section controls which plots will be generated and how they will be displayed (Figure 9).

This section includes options for:

- heatmaps;
- volcano plots;
- PCA plots;
- PCA sample labels;
- heatmap scale method;
- volcano labels;
- cutoff lines.

The screenshot shows the 'Settings Builder' tab in a web application. At the top, there are navigation links: 'Data - Overview', 'Settings Builder' (active), 'Pipeline Log', and 'Results Gallery'. Below these, there are two sub-tabs: 'Settings form' (active) and 'Variable guide'. A message states: 'Use the form below to configure key variables with standardized input fields. The variable guide is in the other tab.' Below this message, there is a 'Settings form' section with a note 'Current values are applied when running the pipeline.' and a blue button 'Save config/settings.R from form'. Underneath, there are three sub-sections: 'Run setup', 'Visual outputs' (active), and 'Exports'. The 'Visual outputs' section contains three main panels: 'Heatmap', 'Volcano', and 'PCA'. The 'Heatmap' panel has four settings: 'Heatmap top N' (input field with '50'), 'Heatmap by model' (dropdown with 'TRUE'), 'Heatmap by model and sex' (dropdown with 'TRUE'), and 'Heatmap scale method' (dropdown with 'zscore'). The 'Volcano' panel has three settings: 'Pairwise volcano plots' (dropdown with 'TRUE'), 'Add volcano labels' (dropdown with 'TRUE'), and 'Add cutoff lines' (dropdown with 'TRUE'). The 'PCA' panel has two settings: 'PCA scaling' (dropdown with 'pareto') and 'Label PCA samples' (dropdown with 'TRUE').

Figure 9. Visual output settings for heatmaps, volcano plots, and PCA plots.

- **Exports**

The **Exports** section controls the output level (Figure 10).

The recommended option for most users is: **Standard**. This keeps the final statistics, principal or combined PCA plots, primary volcano plots, README, and log files.

The **Full/Debug** option generates more intermediate and technical files. It is useful for debugging, but it may create many outputs.

After reviewing the settings, click: **Save config/settings.R from form**.

This creates or updates a local **settings.R** file based on the selected options. This file is used during pipeline execution but is not intended to be committed to the public repository.

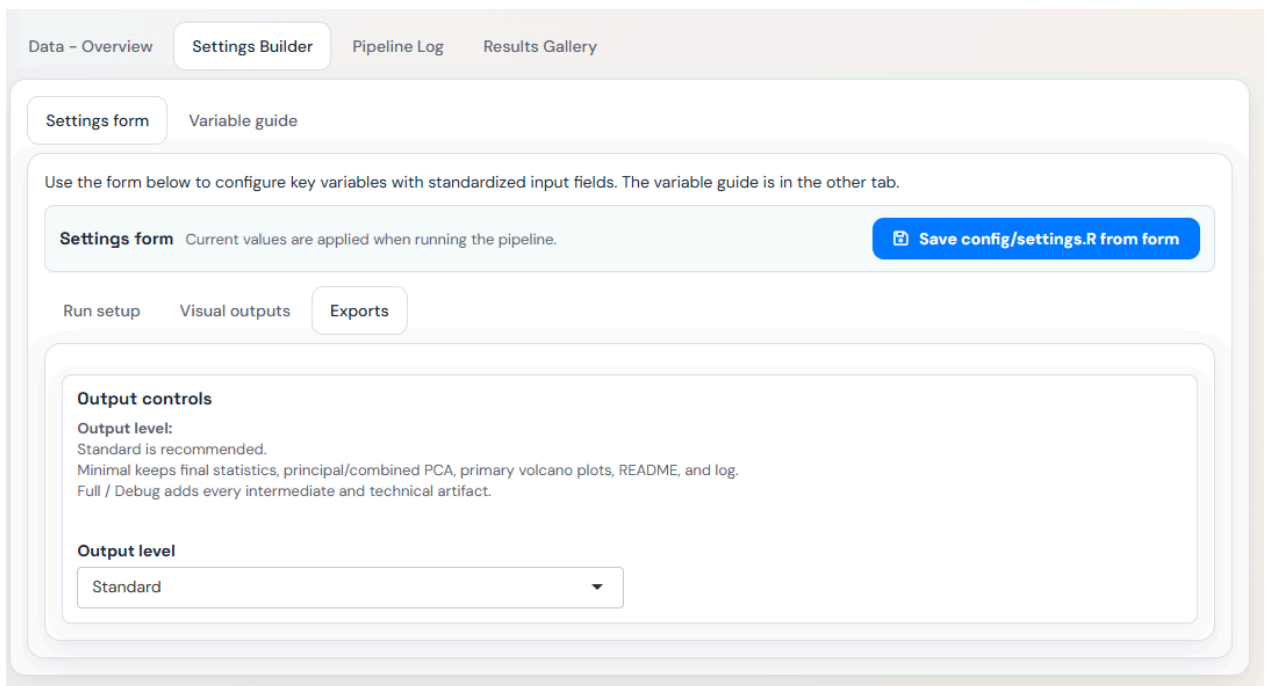


Figure 10. Export level configuration in the Settings Builder.

9.7 STEP 7 — RUN THE PIPELINE

After uploading files and reviewing the settings, click: ***Run pipeline***

The status area should indicate that the pipeline has started.

Depending on the dataset size and the selected settings, execution may take from a few minutes to more than one hour.

Avoid closing RStudio or the application while the pipeline is running.

9.8 STEP 8 — INSPECT THE PIPELINE LOG

Open the Pipeline Log tab to monitor execution (Figure 11).

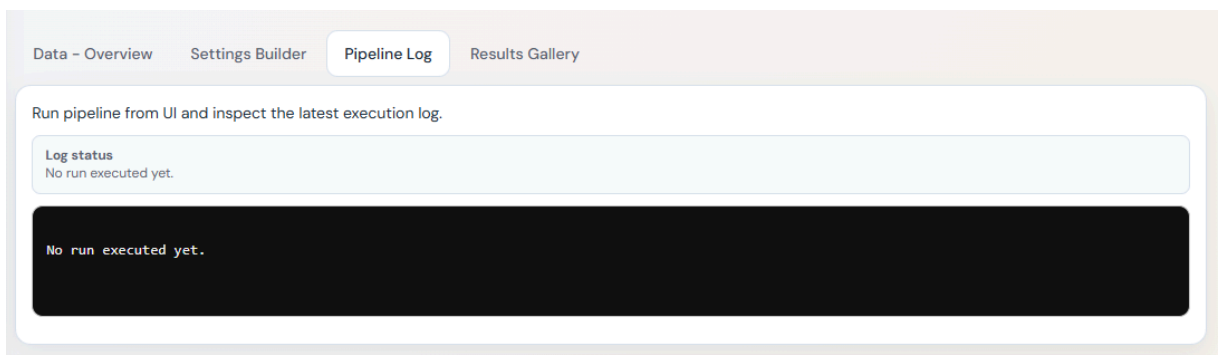


Figure 11. Pipeline Log tab before pipeline execution.

The log can help identify:

- whether the pipeline started correctly;
- which step is currently running;
- warnings;
- missing files;
- package problems;
- metadata problems;
- normalization or filtering issues;
- whether the pipeline completed successfully.

If the pipeline fails, the **Pipeline Log** should be checked before changing the input files or settings.

9.9 STEP 9 — INSPECT THE RESULTS GALLERY

After the pipeline finishes, open the **Results Gallery** tab (Figure 12).

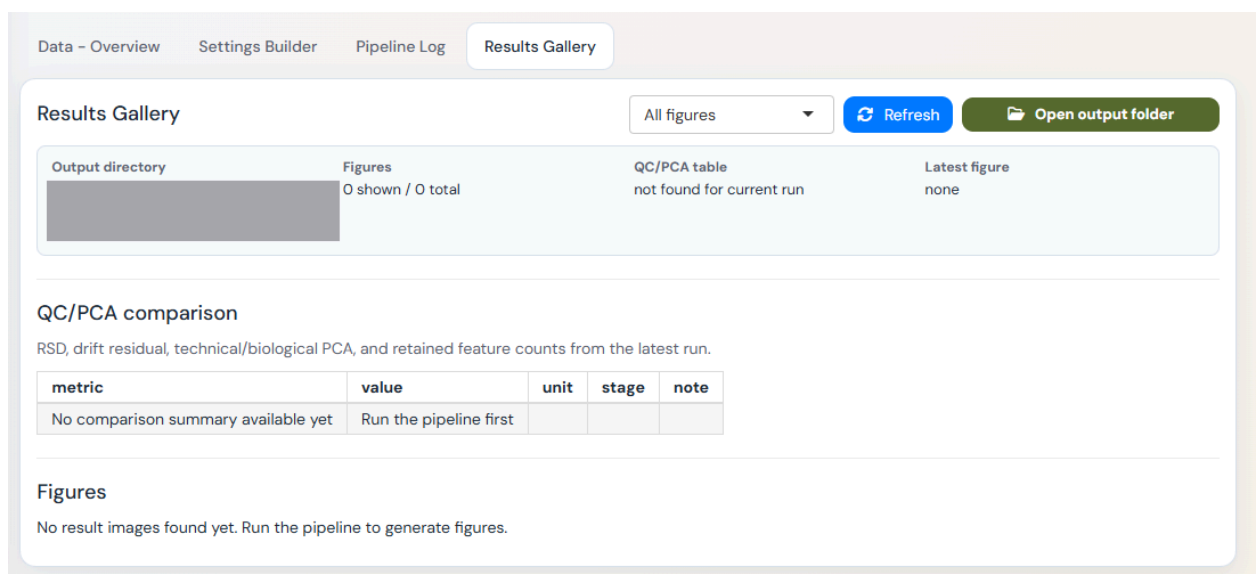


Figure 12. Results Gallery tab displaying generated summaries and visual outputs after pipeline execution.

This tab displays selected result summaries and generated figures, such as:

- PCA plots;
- volcano plots;
- heatmaps;
- normalization comparison plots;
- QC/PCA comparison summaries.

The **Results Gallery** also provides access to the output folder. If no results are displayed, click **Refresh** or confirm that the pipeline completed successfully.

9.10 STEP 10 — OPEN THE OUTPUT FOLDER

After the run is complete, open the output folder to inspect all generated files.

Recommended files and folders to inspect first:

1. PIPELINE_LOG.txt
2. README.txt
3. audit files;
4. normalization and QC summaries;
5. PCA plots;
6. volcano plots;
7. heatmaps;
8. statistical result tables;
9. MetaboAnalyst-compatible export files.

The app provides a visual summary, but the complete set of results is saved in the output directory.

10. RUN THE PIPELINE FROM CODE

CD-MetaboRefine can also be executed directly from R scripts, without using the Shiny graphical interface.

This execution mode is recommended for:

- reproducible analyses;
- advanced users;
- debugging;
- batch processing;
- re-running the same analysis with the same configuration;
- users who prefer to edit configuration files manually.

In this mode, the pipeline is controlled by a local configuration file created from the template: *pipeline/config/settings.R*;

The repository provides *settings.example.R* as the configuration template. Before running the pipeline from code, the user creates a local *settings.R* file from this template.

The workflow is then executed using the main script: *pipeline/run_pipeline.R*

10.1 STEP 1 — CREATE THE LOCAL SETTINGS FILE

Before running the pipeline from code, create a local configuration file based on the example template.

1. Navigate to *pipeline/config/*
2. Copy the example file: *settings.example.R*
3. and rename the copy to *settings.R*

The file *settings.example.R* is the template provided with the repository and should remain unchanged.

The file *settings.R* is the local configuration file used for a specific analysis. It stores local input paths, output paths, selected groups, normalization options, filtering parameters, statistical thresholds, and export settings.

Because *settings.R* may contain user-specific paths and analysis settings, it should not be committed to the public repository.

IMPORTANT:

- *settings.R* is a local configuration file and should not be committed to the public repository.

10.2 STEP 2 — CONFIGURE THE MAIN OPTIONS

Open `pipeline/config/settings.R` in RStudio and review the main options before execution.

The most important options to check are:

- input data file path;
- metadata file path;
- optional reference file path;
- output directory;
- selected metadata groups;
- weight normalization;
- preprocessing or normalization scenario;
- QC/normalization audit;
- RSD filter and threshold;
- duplicate handling strategy;
- p-value cutoff;
- FDR cutoff;
- FC cutoff;
- PCA settings;
- heatmap settings;
- volcano plot settings;
- export level.

Use valid file paths and confirm that the input files exist before running the pipeline.

When possible, use paths inside the project folder or simple absolute paths. Avoid very long paths and paths containing special characters.

10.3 STEP 3 — EXECUTE THE PIPELINE FROM RSTUDIO

After editing and saving *settings.R*, open the RStudio Console and run:

```
</> R
```

```
source("pipeline/run_pipeline.R")
```

This command starts the complete pipeline using the options defined in *settings.R*.

During execution, messages will appear in the RStudio Console. These messages help monitor the progress of the analysis and identify possible warnings or errors.

10.4 STEP 4 — EXECUTE THE PIPELINE FROM A TERMINAL

The pipeline can also be executed from a system terminal.

From the project root directory, run:

```
</> PowerShell
```

```
Rscript pipeline/run_pipeline.R
```

This option is useful for advanced users, automated workflows, or remote execution.

If the terminal does not recognize Rscript, run the pipeline from RStudio instead or check whether R was correctly added to the PATH system.

10.5 STEP 5 — READ THE PIPELINE LOG

During execution, CD-MetaboRefine writes a detailed log file to the output directory.

The main log file is usually named: *PIPELINE_LOG.txt*

This file records:

- selected settings;
- input file paths;
- executed steps;
- filtering summaries;
- normalization messages;
- warnings;
- errors;
- completion status.

If an error occurs, inspect *PIPELINE_LOG.txt* before changing the input files or settings.

The log is one of the most important files for debugging because it records what the pipeline did during the run.

10.6 STEP 6 — INSPECT THE GENERATED OUTPUTS

After the script finishes, open the selected output directory and inspect the generated files.

Recommended files and folders to review first:

1. *PIPELINE_LOG.txt*
2. *README.txt*
3. audit files;
4. normalization summaries;
5. PCA plots;
6. volcano plots;
7. heatmaps;
8. statistical result tables;
9. MetaboAnalyst-compatible export files.

The exact output structure depends on the selected settings and export level.

10.7 WHEN TO USE THE SCRIPT-BASED MODE INSTEAD OF THE SHINY APP

The Shiny application is recommended for interactive use and visual configuration. The script-based mode is recommended when:

- the same analysis needs to be repeated exactly;
- the user wants to keep a clear record of all settings;
- many datasets need to be processed;
- debugging is required;
- the pipeline is being modified or extended;
- the user wants to run the workflow outside the Shiny interface.

Both execution modes use the same underlying pipeline modules. The difference is only how the configuration is prepared and how the workflow is started.

11. UNDERSTAND THE OUTPUT STRUCTURE

After CD-MetaboRefine finishes running, the pipeline creates an output directory containing processed data, audit files, statistical results, plots, logs, and export files.

The exact content of the output folder depends on the selected settings, input files, comparisons, normalization method, and export level.

By default, the output directory is usually named: *output/*

However, the user can select a different output directory from the Shiny interface or from the *settings.R* configuration file.

11.1 MAIN OUTPUT FILES

The most important files to inspect after each run are summarized in Table X.

Table X. Main output files generated by CD-MetaboRefine.

File or folder	Purpose
<i>PIPELINE_LOG.txt</i>	Records execution messages, warnings, selected settings, and possible errors.
<i>README.txt</i>	Summarizes the run configuration and output organization.
<i>audits/</i> or <i>audits_global/</i>	Contains quality-control, filtering, duplicate handling, and normalization of audit files.
<i>exports/</i> or <i>exports_global/</i>	Contains processed tables and files for downstream analysis.
<i>plots/</i> or <i>plots_global/</i>	Contains generated figures such as PCA plots, volcano plots, heatmaps, and normalization diagnostics.
<i>stats/</i>	Contains statistical result tables for the selected comparisons.
<i>metaboanalyst export files</i>	Contains tables formatted for downstream analysis in MetaboAnalyst.

The first file to open should usually be *PIPELINE_LOG.txt*, because it confirms whether the pipeline finished successfully.

11.2 TYPICAL OUTPUT ORGANIZATION

The output folder may contain global results and model-specific results.

A typical output structure may look like this:

```
output/
├── PIPELINE_LOG.txt
├── README.txt
├── global/
│   ├── audits_global/
│   ├── exports_global/
│   └── plots_global/
├── model-X/
│   ├── exports/
│   ├── plots/
│   └── stats/
└── model-Y/
    ├── exports/
    ├── plots/
    └── stats/
```

The global/ folder usually contains outputs calculated across the full dataset or shared across models.

Model-specific folders contain outputs generated separately for each experimental model.

11.3 RECOMMENDED FILES TO REVIEW FIRST

After each run, inspect the outputs in the following order:

1. **PIPELINE_LOG.txt**
Check whether the pipeline completed successfully and whether warnings were reported.
2. **README.txt**
Review the configuration summary and output organization.
3. **Audit files**
Check sample matching, metadata alignment, filtering behavior, normalization summaries, duplicate handling, and QC information.
4. **Normalization and QC diagnostic plots**
Inspect whether preprocessing improved technical consistency or reduced analytical drift.

5. **PCA plots**

Review sample distribution, clustering patterns, QC behavior, and possible outliers.

6. **Statistical result tables**

Inspect p-values, FDR values, fold changes, and selected comparisons.

7. **Volcano plots**

Visualize the relationship between statistical significance and fold change.

8. **Heatmaps**

Inspect abundance patterns across samples and groups.

9. **MetaboAnalyst export files**

Use these files for downstream analysis in MetaboAnalyst when needed.

11.4 AUDIT FILES

Audit files are technical files that help verify whether the pipeline behaved as expected.

They may include information about:

- sample matching;
- metadata alignment;
- missing values;
- filtering steps;
- RSD filtering;
- duplicate handling;
- normalization settings;
- QC behavior;
- retained and removed features;
- runtime information.

Audit files are important because they help explain why some features were kept, removed, transformed, or exported.

11.5 STATISTICAL RESULT TABLES

Statistical result tables summarize the results of the selected comparisons.

They usually include information such as:

- feature or metabolite name;
- comparison name;
- group direction;
- $\log_2$ fold change;
- p-value;
- FDR-adjusted p-value;
- regulation status;
- annotation information.

The interpretation of $\log_2$ fold changes depending on the order of the comparison.

For example, if the comparison is: TG vs WT; then the direction is interpreted as: $\log_2\text{FC} = \log_2(\text{TG} / \text{WT})$.

- Positive $\log_2\text{FC}$ values indicate higher abundance in TG compared with WT.
- Negative $\log_2\text{FC}$ values indicate lower abundance in TG compared with WT.

11.6 PCA PLOTS

PCA plots are exploratory visualizations used to inspect sample distribution.

They can help identify:

- sample clustering;
- separation between biological groups;
- possible outliers;
- QC sample behavior;
- effects of normalization or preprocessing.

PCA plots should not be interpreted alone as statistical proof. They are useful for exploring general data structure.

11.7 VOLCANO PLOTS

Volcano plots combine fold change and statistical significance in one figure.

In a volcano plot:

- the x-axis usually represents $\log_2$ fold change;
- the y-axis usually represents $-\log_{10}(\text{p-value})$ or $-\log_{10}(\text{FDR})$;
- points farther from zero on the x-axis have stronger fold-change differences;
- points higher on the y-axis have stronger statistical evidence;
- labeled points usually represent selected or significant features.

Volcano plots are useful for identifying candidate metabolites that combine statistical significance and biological effect size.

11.8 HEATMAPS

Heatmaps show abundance patterns across samples and selected features.

They are useful for visualizing:

- whether samples cluster by group;
- whether selected metabolites show similar patterns;
- whether the signal is consistent across samples;
- whether specific groups show higher or lower abundance patterns.

Heatmaps are usually generated using scaled values, such as z-scores or Pareto-scaled values. This means that the colors represent relative abundance patterns, not always raw intensity values.

11.9 NORMALIZATION AND QC DIAGNOSTIC PLOTS

Normalization and QC diagnostic plots help evaluate whether preprocessing improved the technical quality of the dataset.

These plots may include:

- normalization comparison plots;
- QC/PCA comparison summaries;
- RSD summaries;
- drift correction diagnostics;
- retained feature counts;
- technical versus biological PCA comparisons.

These outputs are especially important when QC-based correction methods are selected, such as QC-LOESS or QC-RSC.

11.10 EXAMPLE VISUAL OUTPUTS

CD-MetaboRefine can generate several types of visual outputs, including PCA plots, volcano plots, heatmaps, and normalization comparison plots (Figure 13).

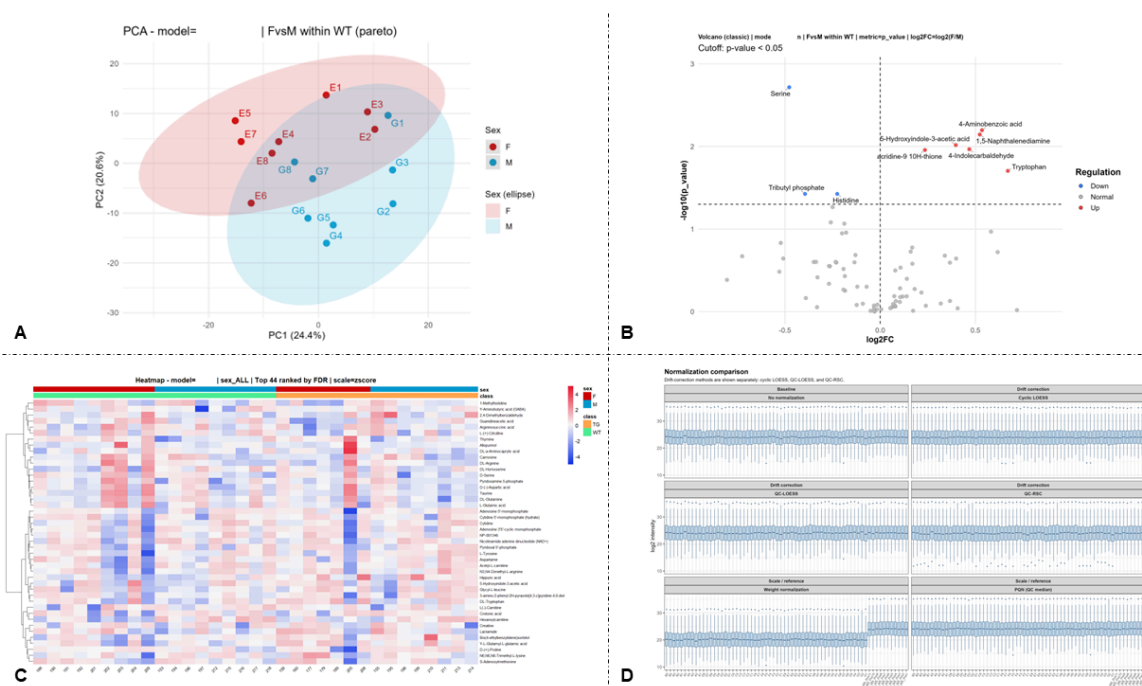


Figure 13. Example visual outputs generated by CD-MetaboRefine. (A) PCA plot used to inspect sample distribution; (B) volcano plot used to visualize statistical significance and fold change; (C) heatmap used to compare feature abundance patterns across samples; and (D) normalization comparison plot used to inspect preprocessing and QC-based correction methods.

11.11 IMPORTANT INTERPRETATION NOTE

The output files should be interpreted together.

A statistically significant result is more reliable when it is supported by:

- consistent preprocessing behavior;
- acceptable QC performance;
- reasonable PCA structure;
- appropriate filtering;
- clear fold-change direction;
- biologically meaningful interpretation.

For this reason, users should not inspect only the final statistical table. The log, audit files, PCA plots, normalization diagnostics, volcano plots, and heatmaps should be reviewed together.

12. TROUBLESHOOTING

This section summarizes common problems that may occur during installation, configuration, file upload, pipeline execution, or result generation.

When an error occurs, always check the following first:

1. the RStudio Console;
2. the CD-MetaboRefine Pipeline Log tab;
3. the PIPELINE_LOG.txt file in the output directory;
4. the input files and metadata file;
5. the selected settings in the Settings Builder.

Most problems are caused by missing packages, incorrect file paths, mismatched sample names, missing metadata columns, or unavailable written permissions in the output directory.

12.1 THE APPLICATION DOES NOT OPEN

Problem:

CD-MetaboRefine does not start after running:

```
</> R
```

```
shiny::runApp("app")
```

Possible causes:

- one or more startup packages are missing;
- the project was not opened from the correct folder;
- the app folder is missing or was moved;
- RStudio is not using the expected R version.

Recommended solution:

Install the startup packages:

```
</> R
```

```
install.packages(c("shiny", "shinyjs", "bslib"))
```

Then restart RStudio and try launching the app again.

Also confirm that the project root contains the app/ folder.

12.2 MISSING PACKAGES

Problem:

The app or pipeline reports that one or more packages are missing.

Possible symptoms:

- the interface does not open;
- the pipeline stops during execution;
- the Console shows messages such as there is no package called....

Recommended solution:

First, install the startup packages:

```
</> R
```

```
install.packages(c("shiny", "shinyjs", "bslib"))
```

Then use the Package Management section inside the app, if available, to install the remaining missing packages.

If automatic installation fails, install the main packages manually:

```
</> R
```

```
install.packages(c(
  "tidyverse", "readr", "readxl", "openxlsx",
  "pheatmap", "ggrepel", "stringi", "RColorBrewer",
  "processx", "shinyFiles", "magick", "BiocManager"
))
```

For Bioconductor packages:

```
</> R
```

```
BiocManager::install(c("pmp", "limma"), ask = FALSE, update = FALSE)
```

After installing packages, restart RStudio.

12.3 *settings.R* NOT FOUND

Problem:

The pipeline cannot find the local configuration file: *pipeline/config/settings.R*

Cause:

The repository includes only *settings.example.R*. The local *settings.R* file has not been created yet.

Recommended solution:

Copy the example configuration file and rename it:

```
</> R
```

```
file.copy(
  "pipeline/config/settings.example.R",
```

```
"pipeline/config/settings.R",  
overwrite = FALSE)
```

Then open settings.R and edit the local paths and analysis options.

If the pipeline is run from the Shiny interface, click: **Save config/settings.R from form** before running the pipeline.

12.4 FILE READ ERROR

Problem:

The pipeline cannot read the data file, metadata file, reference file, or injection order file.

Possible causes:

- invalid file path;
- unsupported file format;
- corrupted spreadsheet;
- file open in Excel;
- missing sheet;
- missing header;
- merged cells;
- special characters in the file path;
- very long folder path.

Recommended solution:

Check that:

- the file exists;
- the file is saved and closed;
- the file extension is supported;
- the file is not corrupted;
- the spreadsheet does not contain merged cells;
- the file path is not too long;
- the file name does not contain unusual characters.

Supported formats include:

- | | |
|---------|--------|
| • .xlsx | • .csv |
| • .xls | • .tsv |

12.5 SAMPLE NAMES DO NOT MATCH

Problem:

The app or pipeline cannot match samples between the Compound Discoverer table and the metadata file.

Possible causes:

- sample names are written differently;
- extra spaces are present;
- sample names contain hidden characters;
- some samples are missing from the metadata file;
- metadata contains samples that are not present in the data file;
- Compound Discoverer sample columns were renamed manually.

Recommended solution:

Check that the sample names match exactly between:

1. the Compound Discoverer feature table;
2. the metadata file;
3. the injection order file, when used.

Example (Table XI):

Table XI. Sample name matching examples.

Compound Discoverer sample	Metadata sample	Correct?
S01"	S01	Yes
S01	s01	No
S01	S01[space]	No
Sample_01	S01	No

Remove extra spaces and standardize sample names before running the pipeline again.

12.6 MISSING METADATA COLUMNS

Problem:

The pipeline reports that one or more metadata columns are missing.

Possible causes:

- the metadata file does not contain the expected column names;
- columns were renamed;
- the wrong metadata file was uploaded;
- the header row is not correctly formatted.

Recommended solution:

Check that the metadata file contains the required or recommended columns for the selected analysis.

Typical columns include:

- sample
- group
- sex
- model
- type
- weight_mg

If weight normalization is enabled, the metadata file must contain a valid weight column.

If QC-based correction is enabled, QC samples must be clearly identified.

12.7 WEIGHT NORMALIZATION ERROR

Problem:

The pipeline fails or gives warnings during weight normalization.

Possible causes:

- the weight column is missing;
- weight values are empty;
- weight values are zero;
- weight values are not numeric;
- QC samples were incorrectly assigned weight values;
- weight normalization was enabled by mistake.

Recommended solution:

Check the metadata file and confirm that the weight column is present and numeric.

If the analysis does not require weight correction, disable weight normalization in the Settings Builder.

12.8 QC-BASED NORMALIZATION DOES NOT WORK

Problem:

QC-LOESS, QC-RSC, or another QC-based correction does not run as expected.

Possible causes:

- QC samples are missing;
- QC samples are not labeled correctly;
- injection order is missing;
- injection order is incorrect;
- too few QC samples are available;
- QC samples are not distributed across the analytical run.

Recommended solution:

Check that:

- QC samples are present in the dataset;
- QC samples are correctly labeled in the metadata file;
- injection order is available when required;
- the injection order reflects the real analytical sequence;
- QC samples are spread across the run.

The injection order can be prepared manually or exported from the Compound Discoverer **Input Files** tab when that table reflects the acquisition sequence.

12.9 NO OUTPUT FILES WERE GENERATED

Problem:

The pipeline appears to run, but no result files are found.

Possible causes:

- wrong output directory selected;
- no write permission in the selected folder;
- pipeline stopped before completion;
- output folder was created in another location;
- export level settings are too restrictive;
- app was closed before the run finished.

Recommended solution:

Check:

1. the selected output directory;
2. the Pipeline Log tab;
3. the PIPELINE_LOG.txt file;
4. the RStudio Console;
5. whether the run finished successfully.

Use a simple output directory such as *output/* inside the project folder.

12.10 RESULTS GALLERY IS EMPTY

Problem:

The Results Gallery tab does not show generated figures.

Possible causes:

- the pipeline has not been run yet;
- the run failed before figures were generated;
- the selected output directory is incorrect;
- the Results Gallery needs to be refreshed;
- the export level did not generate the expected image.

Recommended solution:

Check that the pipeline was completed successfully. Then click **Refresh**, if available, and confirm that the output folder contains plot files.

Also inspect the output folder manually.

12.11 RScript NOT RECOGNIZED

Problem:

The terminal cannot detect the Rscript executable when running:

```
</> R
```

```
Rscript pipeline/run_pipeline.R
```

Cause:

R is not available in the PATH system.

Recommended solution:

Use RStudio instead:

```
</> R
```

```
source("pipeline/run_pipeline.R")
```

Advanced users can also add the R installation directory to the operating system PATH.

On Windows, the Rscript executable is usually located inside the R installation folder, for example: **C:/Program Files/R/R-x.x.x/bin/Rscript.exe**

12.12 GENERAL DEBUGGING CHECKLIST

If the error is unclear, follow this checklist:

1. Restart RStudio.
2. Reopen the project from the project root folder.
3. Confirm that the *app/* and *pipeline/* folders are present.
4. Confirm that required packages are installed.
5. Check that input files are closed in Excel.
6. Confirm that sample names match.
7. Confirm that metadata columns are present.
8. Confirm that QC samples are correctly labeled when needed.
9. Confirm that injection order is correct when QC-based correction is selected.
10. Confirm that the output directory is writable.
11. Run the pipeline again.
12. Inspect the **Pipeline Log** and *PIPELINE_LOG.txt*.

Most errors can be identified by carefully reading the Console messages and the pipeline log.